

Chapter 1

INTRODUCTION

1.1 INTRODUCTION

Natural Language Processing (NLP) is built upon the intersection of linguistics, computer science, and artificial intelligence. Its goal is to enable computers to understand, interpret, and generate human language in a manner that mimics human-like comprehension. At its core, NLP seeks to decipher the meaning, context, and intent behind human language. It grapples with the nuances of language, such as ambiguity, context-dependence, and cultural variations. NLP leverages a plethora of techniques and algorithms, including statistical methods, rule-based systems, machine learning, and deep learning. These approaches enable NLP systems to learn patterns from large amounts of text data and make predictions or generate text based on learned patterns. Despite significant advancements, NLP grapples with several challenges. These include dealing with language ambiguity, understanding context, handling rare or unseen words, addressing biases in data, and achieving cross-lingual understanding. The applications of NLP are vast and diverse. They span across industries such as healthcare (clinical documentation, patient monitoring), finance (sentiment analysis, fraud detection), e-commerce (recommendation systems, customer support), education (automated grading, language learning), and more. NLP powers virtual assistants, chatbots, language translation services, and sentiment analysis tools, among others. As NLP systems become more pervasive, ethical considerations come to the forefront. Issues such as bias in language models, privacy concerns, and the responsible use of NLP technology necessitate careful attention from researchers, developers, and policymakers. The field of NLP is evolving rapidly, driven by advancements in machine learning, deep learning, and computational resources. Future directions include improving the robustness and interpretability of NLP models, addressing societal and ethical implications, and pushing the boundaries of language understanding and generation.

NLP spans a broad spectrum of tasks, ranging from fundamental processes like tokenization and part-of-speech tagging to more intricate ones such as sentiment analysis, language translation, and question-answering. Each task serves to tackle distinct facets of language understanding and generation. Within this landscape, Grammatical Error Correction stands out as a crucial task aimed at rectifying grammatical errors within the text.

Grammar Error Correction (GEC) refers to the task of detecting and correcting errors in grammar of a grammatically incorrect sentence. These errors can include all kinds of grammatical errors like spelling mistakes, incorrect use of articles, prepositions, pronouns, nouns, etc or even poor sentence construction. GEC is essentially an NLP task, which makes our machines think like us, by learning how to correct grammar like any human would. There are a wide range of applications grammatical error correction is used in, which test the grammar of the input text, for example, in email applications, text editors, notepads, etc.

The GEC model for Kannada is a pioneering initiative integrating Natural Language Processing (NLP) technology to enhance the quality of written Kannada language. NLP, a branch of artificial intelligence, empowers the project to delve deep into the intricacies of Kannada grammar, syntax, and vocabulary, offering a robust and intelligent language processing solution.

1.2 CHALLENGES AND OPEN ISSUES IDENTIFIED

- **Contextual Understanding:** Many grammatical errors depend on the context in which a sentence is used. Understanding the context is crucial for accurate error correction. Developing models that can grasp the broader context of a sentence to make more accurate corrections.
- **Ambiguity Resolution:** Ambiguities in language can lead to incorrect corrections. Resolving ambiguity is challenging, especially when multiple corrections are possible.

- **Non-Standard Language and Creativity:** Some users intentionally use non-standard language or employ creative expressions, making it difficult for correction tools to distinguish between errors and intentional language use. Developing models that can recognize and respect intentional deviations from standard grammar while still providing helpful suggestions.
- **Handling Style Variations:** Writing styles vary widely, and what may be considered correct in one style may be incorrect in another. Correction tools should be sensitive to diverse writing styles.
- **Limited Availability of Training Data:** High-quality, diverse training data is essential for training effective grammatical error correction models. Collecting and curating large and diverse datasets to train models that generalize well across various writing styles, genres, and domains.
- **Idiomatic Expressions and Colloquialisms:** Correcting idiomatic expressions and colloquialisms can be challenging, as literal corrections may result in unnatural language.

1.3 PROBLEM STATEMENT

Kannada has a unique linguistic structure, and publicly no Kannada grammar-correcting tools are available. Hence development of a tool to fix spelling and grammar errors in Kannada documents is proposed in this project.

1.4 OBJECTIVES

- Creation of Kannada lexicon for spell-checking.
- Creation of a labelled dataset for grammatical error detection.
- Spell checker: Detects misspelt words and offers suggestions, ensuring accurate spellings.
- Grammatical error detection using the BERT model.
- Grammatical error correction by identifying and correcting grammatical errors based on subject-verb agreement.

1.5 SCOPE

The project scope involves developing a grammar error correction tool specifically for Kannada documents. This tool will focus on identifying and correcting common grammatical errors, including spelling mistakes, punctuation errors, and syntax issues. It will provide a user-friendly interface for inputting text and viewing corrections, with options for real-time feedback and error handling. Special attention will be given to Kannada language-specific features and linguistic nuances. Deployment options will include standalone applications, web-based tools, and integration capabilities. Continuous improvement and future expansion will be prioritized based on user feedback and evolving language trends.

1.6 WORK PLAN

- Problem Scope: We define the problem statement and gather the requirements to solve the problem.
- Understanding The Problem: Prepare the initial synopsis regarding the feasibility study and define the objectives.
- Creation of dataset set: We created our own dataset for this project which consists of 45000 sentences.
- Planning And Brainstorming: Discuss the feature implementation and prepare SRS documentation along with functional and non-functional requirements.
- Designing: Divide the project into different modules. Prepare deadlines for each module along with their implementation.
- Implementation: Code the designed modules and prepare necessary models.
- Testing And Refactoring: Test for accuracy of the built models, Refactor the code necessary, and make the code clean with standard file structure.
- Documentation: Record the progress of the project and guide the users, detailing how to use our project.

Chapter 2

LITERATURE REVIEW

2.1 INTRODUCTION

The development of spell checkers and grammar checkers for Kannada documents necessitates a comprehensive understanding of linguistic structures, computational linguistics methodologies, and existing research in natural language processing (NLP). This literature survey aims to explore the landscape of relevant literature, methodologies, and technologies to inform the design and implementation of robust language processing tools tailored for Kannada.

The literature survey encompasses a broad spectrum of research areas pertinent to spell checking and grammar checking systems, including natural language processing, computational linguistics, machine learning, and linguistics. By critically examining existing literature, methodologies, and technologies, this survey aims to identify key insights, challenges, and opportunities in the development of language processing tools specifically tailored for Kannada.

The survey begins by reviewing foundational concepts and theories in linguistics and computational linguistics, providing a theoretical framework for understanding the linguistic structures and complexities inherent to the Kannada language. Subsequently, it explores existing research and methodologies in the domain of spell checking and grammar checking systems, encompassing both rule-based and data-driven approaches.

Furthermore, the survey examines relevant datasets, corpora, and resources available for Kannada language processing, evaluating their adequacy and suitability for training and evaluating spell checkers and grammar checkers. Additionally, it investigates evaluation metrics and methodologies employed in assessing the performance and effectiveness of language processing tools, aiming to establish standards for benchmarking and comparison.

Through a comprehensive synthesis of existing literature and methodologies, this survey endeavors to inform the design, development, and evaluation of spell checkers and

grammar checkers tailored for Kannada documents. By identifying gaps, challenges, and opportunities, this survey aims to provide a foundation for future research endeavors in the domain of Kannada language processing, facilitating the advancement of linguistic technology and accessibility of digital content in Kannada.

2.2 LITERATURE STUDIES

The paper introduces an innovative approach to spell checking for highly inflectional Indian languages, aiming to address challenges in post-OCR error detection and correction. Utilizing Long Short-Term Memory (LSTM) models tailored for character-level language modeling, the study focuses on OCR documents in Sanskrit, Malayalam, Kannada, and Hindi, comprising over 352,000 word pairs. The LSTM model, incorporating a fixed delay to consider surrounding character sequences, is trained using one-hot encoded characters from OCR words and aligned ground truth words. Despite the manual effort required to create accurate ground truth datasets and potential challenges in correcting real word errors, the proposed approach demonstrates significant improvements. The LSTM-based model achieves F-Scores above 92.4% and reduces Word Error Rates (WER) by at least 26.7% across languages, outperforming state-of-the-art results in Malayalam and Hindi. Despite limitations, the study provides a robust solution for enhancing OCR accuracy in Indian languages, facilitating more reliable text processing in this domain[1].

The paper presents a hybrid model for improving Urdu spell checking by combining Soundex and Shapex algorithms with other ranking techniques. The aim is to enhance accuracy and efficiency in detecting and correcting spelling errors in Urdu, a language with unique phonetic and visual characteristics. The study evaluates multiple combinations of ranking approaches using a dataset of 147 errors and observes that while Soundex and Shapex excel in matching substitution errors, they struggle with insertion or deletion errors. To address this limitation, hybrid models integrate N-gram and LCS techniques. Results show that the hybrid models outperform standalone methods, with significant improvements in recall, precision, and F1-measure. N-gram-based approaches generally exhibit better recall, while Shapex is more adept at handling visually similar

errors. The best-performing hybrid model achieves an F1-measure of 98.68% for top one suggested candidates, demonstrating the effectiveness of combining different techniques to enhance Urdu spell checking [2].

In "A Context-Sensitive Real-Time Spell Checker with Language Adaptability" (2020) by Prabhakar Gupta, the central focus is on devising a spell checker capable of functioning in real-time and adjusting to diverse languages. The study adopts a comprehensive methodology, leveraging a combination of Wikipedia articles and carefully curated video subtitles to construct dictionaries tailored for different languages. This approach facilitates the system's ability to efficiently detect and rectify spelling errors as they occur. Evaluation of the system reveals impressive precision rates across multiple languages, showcasing its superiority over established tools such as GNU Aspell and Hunspell. However, the research also acknowledges challenges associated with memory requirements, particularly concerning the storage of extensive dictionaries, and variations in performance stemming from differences in dataset quality. Despite these hurdles, the study underscores the promising potential of the proposed approach in advancing spell checking capabilities across a range of linguistic contexts [3].

"Real-word Errors in Arabic Texts: A Better Algorithm for Detection and Correction" (2019) by Aqil M. Azmi, Manal N. Almutery, and Hatim A. Aboalsamh addresses the challenge of identifying and rectifying real-word errors in Arabic texts. Their Java-based system leverages publicly available modules and libraries for text preprocessing, incorporating stemmers like Khoja, Light10, and MADAMIRA. Using the Damerau-Levenshtein distance for error generation and DALM for language modeling, alongside SVM models generated using Weka, the system achieves high accuracy rates, particularly with MADAMIRA stemmer and non-linear SVM kernels like RBF and sigmoid. Despite promising results, limitations include a decline in performance with increased distinct words in a corpus and occasional failure to select the most appropriate correction candidate. Overall, the study offers a comprehensive approach to real-word error detection and correction in Arabic texts, with potential for further refinement in the error correction mechanism to enhance real-world applicability [4].

"A rule-based Bengali grammar checker" (2021) by A.N.M Fahim Faisal, Md. Afikur Rahman, and Tanjila Farah introduces a novel system addressing grammatical errors in Bengali text, filling a gap in existing research focused on spell checking, Text to Speech (TTS), and Optical Character Recognition (OCR). Utilizing a dataset of Bengali sentences, the study trained and validated the system by tokenizing and tagging sentences with Parts of Speech (POS) using the BNLTK toolkit. The system then matched these POS-tagged sentences against predefined Bengali grammar rules, achieving promising accuracy rates of 92% for Set 1, 80% for Set 2, with an average of 86%. However, the reliance on rudimentary POS tags and predefined rules may lead to occasional false positives or negatives, highlighting the need for further refinement to handle complex sentence structures with greater precision for broader applicability [5].

The paper introduces a rule-based grammar and spell-checking approach utilizing Python libraries tailored for Natural Language Processing (NLP), such as NLTK for sentence boundary detection and POS tagging, pyEnchant for spell checking, and Inflect for handling indefinite articles. With a dataset comprising nearly 100 sentences, the study focuses on sentence boundary detection, POS tagging, and spell checking using pyEnchant against a standard dictionary. The Inflect library aids in handling indefinite article errors. The model achieves a 94% accuracy for spell checking and a perfect score for handling indefinite articles, showcasing its effectiveness. Though challenges remain in handling certain names and short forms, a feature allowing users to add original words enhances accuracy by reducing false alarms. Overall, the paper underscores the efficacy of a rule-based approach using Python libraries for grammar and spell checking tasks, yielding promising results in accuracy [6].

The study introduces a novel hybrid approach for detecting and correcting grammatical errors in Sinhala text, aiming to address the language's complexity and the inadequacy of existing grammar checkers. The dataset comprises approximately 800 Sinhala active sentences sourced from online newspapers and validated by language experts. Combining rule-based methods with machine learning algorithms, the approach involves spell-

checking, part-of-speech tagging, and morphological analysis tailored for Sinhala. A decision tree-based machine learning algorithm evaluates sentence correctness, focusing on verbs and subjects, achieving an accuracy rate of 88.6% and an F-measure of 92.47%. Despite successes, challenges remain in handling certain sentence structures and the absence of a comprehensive lexical analyzer for Sinhala. Nonetheless, the research contributes significantly to digital resources for Sinhala, addressing longer sentences and a broader range of grammatical features, enhancing its applicability for real-world tasks[7].

The paper explores an innovative online grammar checker system utilizing a neural network model, specifically the Transformer model, renowned for its capability to identify various grammatical errors in text. Leveraging pre-trained word embeddings and decoder parameters, the study adapts the Transformer model for the Grammar Error Correction (GEC) task, achieving competitive results on the CoNLL-2014 shared task test dataset. The system's design integrates pre-processing techniques like Byte Pair Encoding (BPE) for word segmentation, spaCy tokenization for handling symbols, and a retrained hunspell spellchecker for addressing spelling errors and unknown words. Implemented as a command-line initiated online grammar checker using Python and Django, the system demonstrates promising performance metrics, with potential optimizations such as adjusting beam search sizes to enhance speed without compromising accuracy. Despite computational overhead limitations associated with ensemble model files, the study underscores the transformative potential of neural network models like Transformer in advancing online grammar correction systems, paving the way for future innovations in automated grammatical error detection and correction [8].

The paper introduces a new technique for improving grammatical error detection systems for language learners by combining pairwise training with pre-trained word embeddings from BERT. Unlike previous methods that used batch training, this approach aims to better differentiate between pairs of samples, aiding in error identification. Using three primary datasets and synthetic error texts, the researchers apply various filtering

measures to ensure data quality. The main model employed is BERT, enhanced with an LSTM layer to capture morphological details. During training, batches contain both correct and incorrect versions of the same text, enabling the model to learn various error patterns. Results show leading performance across four standard benchmarks, with enhancements averaging approximately 2.5%. Despite promising progress, challenges include potential noise from back translation in synthetic error creation and increased computational demands for pairwise training compared to batch training. Overall, the paper presents an effective strategy for grammatical error detection, suggesting future improvements in data generation and training efficiency [9].

"Symspell and LSTM-based spellcheckers for Tamil" (2020) by Selvakumar Murugan, Tamil Arasan Bakthavatchalam, and Malaikannan Sankarasubbu presents a thorough investigation into spell checking methods designed for the Tamil language, emphasizing the challenges specific to Indian languages. The study evaluates three spell checkers: bloom-filter, symspell, and an LSTM-based approach, utilizing two primary datasets. While the bloom-filter proved simplistic yet effective for basic spell checking, symspell offered fast correction suggestions at the expense of significant memory requirements. The LSTM-based approach, though innovative, demonstrated limited practicality with an exact match score of only 0.40 and high computational demands. The study discusses the practical implications and limitations of these approaches across different runtime environments, emphasizing the need for further research to develop more robust spell checking tools for Tamil [10].

Chapter 3

SYSTEM REQUIREMENTS

3.1 INTRODUCTION

In the current digital era, efficient communication across a range of platforms and media is essential. The way ideas are communicated and comprehended is greatly influenced by the quality of written content, from academic papers to business emails. But maintaining perfect grammar and syntax can be difficult. To meet this challenge, the spell and grammar-checking tool offers a reliable and user-friendly method for evaluating and enhancing the grammatical accuracy of written material.

The tool will accept user input and offer feedback on spelling and grammar. The system will serve a wide spectrum of users, including professionals, educators, and students, by providing basic spelling and grammar-checking features.

The system requirements specifications outline the hardware and software prerequisites necessary for the proper functioning of the grammar and spell-checking tool. These specifications ensure that users have the necessary hardware configuration and software environment to effectively utilize the tool's features. By adhering to these requirements, users can maximize the tool's performance and functionality, thus enhancing their overall experience with the application. The following sections detail the specific hardware and software requirements needed to run the tool seamlessly on various computing devices and operating systems.

3.2 HARDWARE REQUIREMENTS

The hardware requirements section outlines the necessary physical specifications that a system must meet to effectively run the grammar and spell-checking tool. These requirements ensure that the system possesses adequate processing power and memory to handle the computational demands of the tool. By adhering to these hardware specifications, users can ensure optimal performance and efficiency when utilizing the tool for language analysis and correction tasks.

To effectively run the grammar and spell-checking tool, the following hardware specifications are required:

- A 4-core CPU: The system should be equipped with a CPU featuring at least four cores to ensure efficient processing of language analysis tasks and seamless operation of the tool.
- At least 8GB of RAM: A minimum of 8GB of RAM is necessary to support the computational demands of the tool. Sufficient RAM ensures smooth performance, especially when handling large volumes of text data during language analysis and correction processes.
- A computational device capable of running a modern browser: The hardware must be capable of running a modern web browser to access the user interface of the tool. This requirement ensures compatibility and accessibility across different devices and platforms.

3.3 SOFTWARE REQUIREMENTS

The software requirements section delineates the essential software components and configurations needed for the grammar and spell-checking tool to operate seamlessly. These requirements encompass the programming language, dependencies, and operating system compatibility necessary to support the tool's functionalities. By meeting these software prerequisites, users can ensure compatibility and stability when installing and running the tool on their computing devices.

The software requirements for running the grammar and spell-checking tool include:

- Python 3.7 or higher: The BERT model, a key component of the tool, requires Python 3.7 or higher to be installed on the system. Python serves as the programming language for implementing the tool's functionalities and integrating various NLP libraries and dependencies.

- Python-based dependencies from PyPI: Other Python-based dependencies necessary for the tool's operation are installed from the Python Package Index (PyPI). These dependencies include libraries and frameworks for NLP tasks, web development, and other functionalities essential for the tool's functionality.
- Operating system compatibility: The tool is compatible with various operating systems, including Windows, Linux, and Mac. Users can install and run the tool on their preferred operating system, ensuring flexibility and accessibility across different computing environments.

3.4 FUNCTIONAL REQUIREMENTS

- Spell Error Detection and Correction
 - Input: Kannada text with potential spelling errors.
 - Output: Corrected Kannada text with spelling errors addressed.
 - Description: The system should be able to detect and correct misspelled words in Kannada using the `Hunspell` library.
- Grammar Error Detection and Correction
 - Input: Kannada sentences with potential grammatical errors.
 - Output: Grammatically corrected Kannada sentences.
 - Description: The system should identify and correct grammatical errors in Kannada sentences using a pre-trained BERT model fine-tuned on the custom dataset. The correction process involves using the Masked Language Model (MLM) of BERT to generate alternate sentences and applying a Grammar Error Detection (GED) model for correction suggestions.
 - Description: The system should facilitate the fine-tuning process by initializing the BERT model with pre-trained parameters and updating the model parameters using labeled data from the our dataset.

3.5 NON-FUNCTIONAL REQUIREMENTS

- Performance: The system should provide accurate spell and grammar error detection and correction.

- **Model Training Time:** The fine-tuning process should be efficient. The fine-tuning process should not exceed a specified time limit.
- **System Resource Utilization:** The system should be resource-efficient.
- **Ease of Integration:** The system should be easy to integrate into existing applications or workflows.
- **Accuracy of Grammar Correction:** The grammar correction should maintain semantic meaning.

3.6 TOOLS AND TECHNOLOGIES

In the development of our grammar and spell-checking tool, it is crucial to employ various dependencies and techniques that enable efficient data processing, advanced language analysis, and user-friendly interface design. This section introduces the key dependencies utilized in our project, each playing a pivotal role in enhancing the functionality and performance of the tool. From data manipulation to web interface development, these dependencies form the backbone of our tool, ensuring its effectiveness and usability in real-world applications.

3.6.1 Pandas

The pandas library is an essential dependency utilized in our grammar and spell-checking tool for efficient data manipulation and analysis. Its versatile data structures, particularly DataFrames, provide a powerful framework for handling structured data. With pandas, we can seamlessly load, manipulate, and process data, enabling various operations necessary for the functionality of our tool. Whether it's cleaning and preprocessing textual data or organizing the results of language analysis, pandas facilitates these tasks with ease and efficiency, contributing to the overall effectiveness of our tool.

3.7.2 Transformers

The transformers library significantly enhances the capabilities of our grammar and spell-checking tool by granting access to state-of-the-art pre-trained models for Natural

Language Processing (NLP). These models, trained on vast amounts of textual data, possess advanced language understanding capabilities, making them invaluable for tasks such as text classification, sentiment analysis, and more. By leveraging these models, our tool can perform sophisticated language analysis and correction with a high degree of accuracy and reliability. The diverse collection of models available in transformers empowers us to tailor our tool's functionality to specific language tasks, ensuring its versatility and effectiveness across various linguistic contexts.

3.6.3 Flask

Flask, a lightweight and versatile web framework, serves as the foundation for building the user interface of our grammar and spell-checking tool. Its simplicity and flexibility make it an ideal choice for developing web applications, providing us with the necessary tools to create an intuitive and user-friendly interface. With Flask, we can design interactive web pages where users can input text, view language corrections, and interact with various features of the tool seamlessly. The modular architecture of Flask allows us to organize our code efficiently, ensuring scalability and maintainability as our tool evolves. Overall, Flask empowers us to deliver a smooth and engaging user experience, enhancing the accessibility and usability of our grammar and spell-checking tool.

3.6.4 Hunspell

Hunspell is a robust spell-checking and morphological analysis library that significantly enhances the accuracy and effectiveness of our grammar and spell-checking tool. By leveraging dictionaries and language rules, hunspell enables our tool to identify and correct spelling errors in textual content. Its comprehensive word databases and morphological analysis capabilities allow for accurate detection of misspelled words and suggest corrections based on linguistic rules. Additionally, hunspell supports multiple languages, making it adaptable to diverse linguistic contexts. Integrating hunspell into our tool ensures that the language correction provided is reliable and linguistically accurate, enhancing the overall quality of the user experience.

3.7 CONCLUSION

In conclusion, the System Requirement and Analysis chapter lays the foundation for the development of a robust grammar and spell-checking tool. By outlining the hardware and software prerequisites, as well as functional and non-functional requirements, the chapter ensures the tool's compatibility, performance, and usability. Additionally, the introduction of key tools and technologies such as Pandas, Transformers, Flask, and Hunspell highlights the critical dependencies utilized to enhance the tool's functionality and efficiency. Overall, this chapter provides a comprehensive overview of the requirements and technologies essential for the successful implementation of the grammar and spell-checking tool.

SYSTEM DESIGN

4.1 HIGH LEVEL DESIGN

Below Figure. 4.1 describes the high-level design of proposed project.

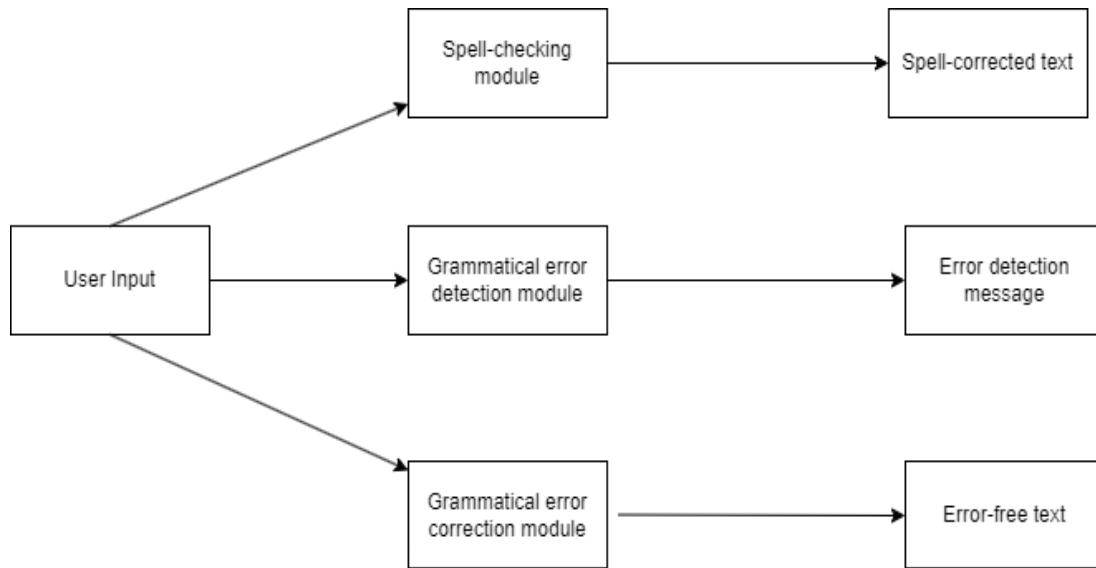


Figure 4.1: High-level design of spell-checker and grammar-checker

Following is the description for Figure. 4.1:

- User Input Module:
 - Receives input text from the user through a user interface.
- Spell-Checking Module:
 - Utilizes a lexicon containing correctly spelled words.
 - Checks each word in the input text against the dictionary to identify misspelled words.
 - Provides suggestions for corrections based on similarity measure (Levenshtein distance).
- Grammatical Error Detection Module:
 - Analyzes the structure and syntax of sentences in the input text.

- Identifies grammatical errors, particularly focusing on subject-verb agreement.
- Grammatical Error Correction Module:
 - Applies linguistic rules and patterns specific to the Kannada to rectify grammatical errors.
 - Focuses on correcting subject-verb agreement errors, tense inconsistencies, and other syntactical issues.
 - Implements rule-based approach.
 - Provides corrected versions of sentences containing grammatical errors.

4.2 LOW-LEVEL DESIGN

4.2.1 Spell-checking and correction

Below Figure. 4.2 describes the low-level design of spell-checker and correction.

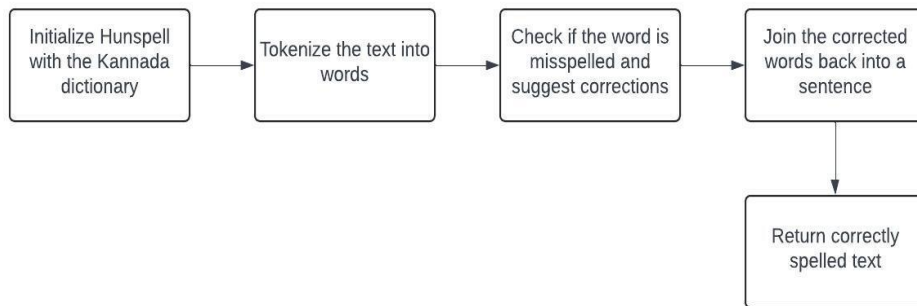


Figure 4.2: Low-level design for Spell-checking and correction

Following is the description for Figure. 4.2:

- Importing Hunspell Library: The code begins by importing the `Hunspell` class from the `hunspell` module. This class is used for spell checking and suggesting corrections.
- Initializing Hunspell: Hunpell object is initialized by providing paths for dictionary and affix files. Input text is provided for spell checking.
- Tokenizing the Text: The input text is tokenized into individual words using the `split()` method. The words are stored in the a list.

- **Checking and Correcting Spelling:** For each word in the tokenized text, check if the word is misspelt. If the word is misspelt, corrections are suggested. If suggestions are available, the first suggestion is taken; otherwise, the original word is kept. The corrected words are stored in a list.
- **Joining Corrected Words:** The corrected words are joined back into a sentence using the `join()` method with space as the separator.
- **Returning Corrected Text:** The function returns the corrected Kannada text.

4.2.2 Grammatical error detection

Below Figure. 4.3 describes the low-level design for Grammatical error detection.

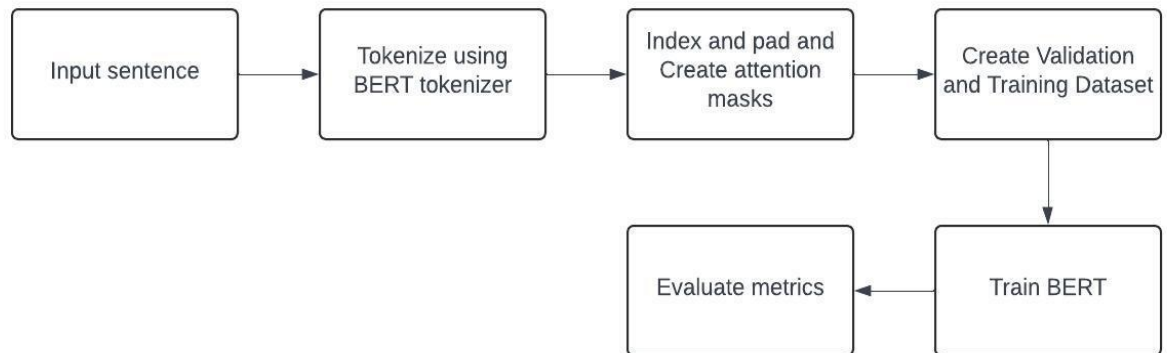


Figure 4.3: Low-level design for Grammatical error detection

Following is the description for Figure. 4.3:

- **Database of Kannada Corpus:**
 - **Implementation:** Create a database containing Kannada words and phrases from various sources.
- **Data Preprocessing using Tokenization:**
 - **Tokenization:** Implement a Kannada-specific tokenizer to break down the text into individual tokens.
 - **Cleaning:** Remove unnecessary characters, punctuation, and handle special symbols unique to Kannada.
- **Divide the Data into Train, Test, and Validation Sets:**

- Splitting: Randomly divide the pre-processed data into training, testing, and validation sets.
- Load BERT Model and Compile for Training Data:
 - Model Loading: Load the DistilBERT model for Kannada.
 - Integration: Integrate the model with appropriate libraries/frameworks (e.g., TensorFlow, PyTorch).
 - Compile: Compile the BERT model with the training data using appropriate loss functions and optimizers.
- Performance Evaluation using Test Data and Freeze the Model:
 - Evaluation: Use the test set to evaluate the performance of the grammatical error detector.
 - Metrics: Calculate relevant metrics (accuracy, precision, recall) to assess the model's effectiveness.
 - Freezing: Save the model's weights and configurations to prevent further training (freezing).

4.2.3 Grammatical error correction

Below Figure. 4.4 describes the low-level design for Grammatical error correction

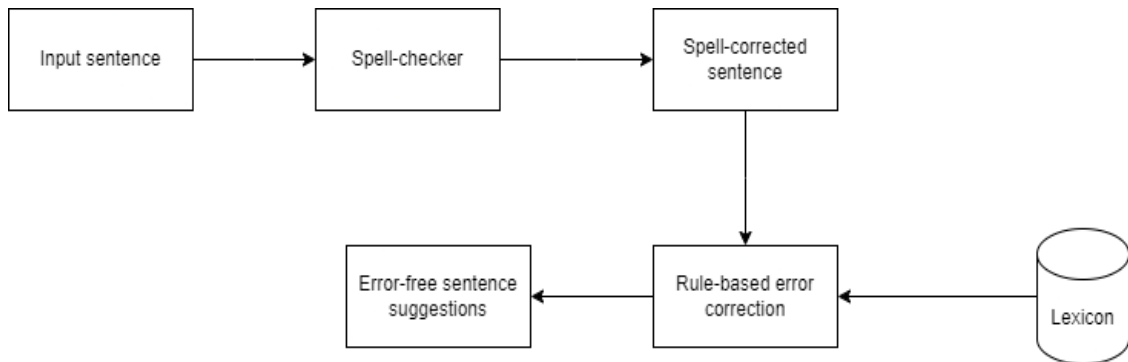


Figure 4.4: Low-level design for Grammatical error correction

Following is the description for Figure. 4.4:

In designing the rule-based approach for grammatical error correction in Kannada, special attention is directed towards addressing subject-verb agreement errors. Subject-verb agreement, a foundational aspect of grammar, ensures that the subject and verb in a sentence align in terms of number and person.

The low-level design of this rule-based approach revolves around leveraging linguistic rules and patterns unique to Kannada to detect and rectify subject-verb agreement discrepancies. This process entails the identification of both the subject and the verb within the sentence, followed by the application of relevant rules to ensure agreement between them.

The diagram 5.4 depicts a rule-based system for Kannada grammatical error correction, specifically focusing on subject-verb agreement errors. Here's a breakdown of the process:

- Input Sentence: The system receives a Kannada sentence as input.
- Spell-checker: The sentence first passes through a spell-checker to identify and rectify any potential spelling mistakes. This ensures the system is working with grammatically correct spellings before proceeding to analyze sentence structure.
- Error-free sentence (Rule-based error correction): The system then employs a rule-based approach to analyze the sentence for subject-verb agreement errors. Kannada, like many other languages, has grammatical rules dictating how verbs conjugate based on the subject's person and number (singular/plural). These rules are encoded within the system.
 - Lexicon: The system relies on a lexicon, which is essentially a large electronic dictionary containing information about Kannada words, including their parts of speech (nouns, verbs, adjectives, etc.). By referencing the lexicon, the system can identify the parts of speech within the sentence.
- Subject-verb agreement: Using the lexicon and the grammatical rules, the system identifies the subject and verb in the sentence. It then checks if the verb conjugation aligns with the subject's person and number. If a mismatch is found, the system suggests potential corrections based on the rules.

- Output: The system outputs the original sentence along with any suggested corrections for subject-verb agreement errors.

Advantages of a Rule-Based Approach

- Transparency: The rules are explicit and can be easily understood and analyzed.
- Efficiency: Rule-based systems can be computationally efficient, especially for well-defined rules.

Disadvantages of a Rule-Based Approach

- Limited Coverage: It can be challenging to capture all the complexities and nuances of natural languages with a finite set of rules.
- Difficulty in Handling Exceptions: The system might struggle to handle exceptions or cases that fall outside the defined rules.

Overall, a rule-based approach offers a straightforward method for Kannada grammatical error correction, particularly for subject-verb agreement. However, it may not be as comprehensive as statistical or machine learning-based approaches for handling the full range of grammatical errors.

SYSTEM IMPLEMENTATION

5.1 SPELL CHECKING AND CORRECTION USING HUNSPELL

Hunspell is an open-source spell-checking and morphological analysis library designed for languages with complex morphology and rich inflections. Originally developed for the Hungarian language, Hunspell has evolved into a versatile tool supporting numerous languages and finding applications in various software applications.

5.1.1 Working methodology of Hunspell

- **Spell Checking:** Hunspell is primarily known for its spell-checking capabilities. It checks the spelling of words against a given dictionary and provides suggestions for corrections when a word is misspelled. This is particularly useful for identifying and correcting typographical errors in written text.
- **Morphological Analysis:** The library goes beyond basic spell-checking by offering morphological analysis. It can break down words into their constituent morphemes, providing insights into word structure, inflections, and grammatical variations. This feature is especially valuable for languages with complex morphological patterns.
- **Affix File Support:** Hunspell uses a combination of dictionary files (`.dic`) and affix files (`.aff`) to perform spell-checking and morphological analysis. The dictionary file contains a list of valid words, while the affix file specifies the rules for word formations, including prefixes, suffixes, and inflections.
- **Multi-Language Support:** One of Hunspell's strengths is its ability to support multiple languages. Users can create or obtain dictionaries and affix files for different languages and use them with the library. This makes Hunspell a popular choice for developers working on applications that require internationalization and support for diverse linguistic contexts.

- **Open Source and Cross-Platform:** Hunspell is released under an open-source license, allowing users to view, modify, and distribute the source code. It is designed to be cross-platform, making it compatible with various operating systems, including Windows, Linux, and macOS.
- **Integration with Software Applications:** Hunspell is widely integrated into software applications, including word processors, text editors, and content management systems. It provides a reliable mechanism for checking and correcting spelling errors in a user-friendly manner. Popular software, such as LibreOffice, Mozilla Firefox, and many others, leverages the capabilities of Hunspell for language checking.

5.1.2 Implementation of spell-checking module

The spell-checker module makes use of the Hunspell library of Python. It compares the input word against a lexicon and looks for affixes in an affix file. The lexicon contains 4879 commonly used Kannada words. Hunspell primarily uses Levenshtein distance as a measure to provide suggestions for misspelt words. Levenshtein distance, also known as edit distance, calculates the minimum number of single-character edits (insertions, deletions, or substitutions) required to change one word into another. When a word is not found in the dictionary, Hunspell searches for similar words within a specified Levenshtein distance and suggests them as potential corrections. These suggestions are based on words in the dictionary that are closest in terms of Levenshtein distance to the misspelt word.

5.2 GRAMMATICAL ERROR DETECTION USING BERT MODEL

BERT (Bidirectional Encoder Representations from Transformers) is a NLP Model proposed by researchers at Google Research in 2018. When it was proposed it achieved state-of-the-art accuracy on many NLP tasks.

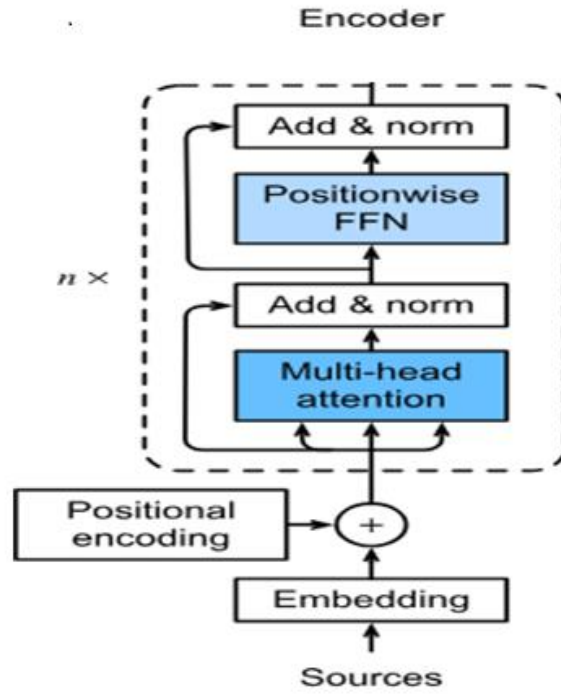


Figure 5.1: Working of BERT model

5.2.1 Working methodology of the BERT model

In essence, BERT is a transformer architecture encoder stack. An encoder-decoder network with a transformer design employs attention on the decoder side and self-attention on the encoder side. The encoder stacks of BERTBASE and BERTLARGE have 12 and 24 layers, respectively. These surpass the Transformer design (6 encoder layers) as stated in the original study. In comparison to the Transformer design proposed in the original study, BERT architectures (BASE and LARGE) also contain more attention heads (12 and 16 respectively) and larger feedforward networks (768 and 1024 hidden units, respectively). It has eight attention heads and 512 hidden units. Whereas BERTLARGE has 340M parameters, BERTBASE has 110M.

The CLS token is the first input used in this model, and then a string of text. A classification token in this case is CLS. The input is then sent to the layers above. After applying self-attention, each layer transfers the output to the following encoder via a feedforward network. A vector of concealed size (768 for BERT BASE) is the output of the model. We may use the output that corresponds to the CLS token to generate a classifier from this model. This trained vector may now be used to many tasks, including translation and classification.

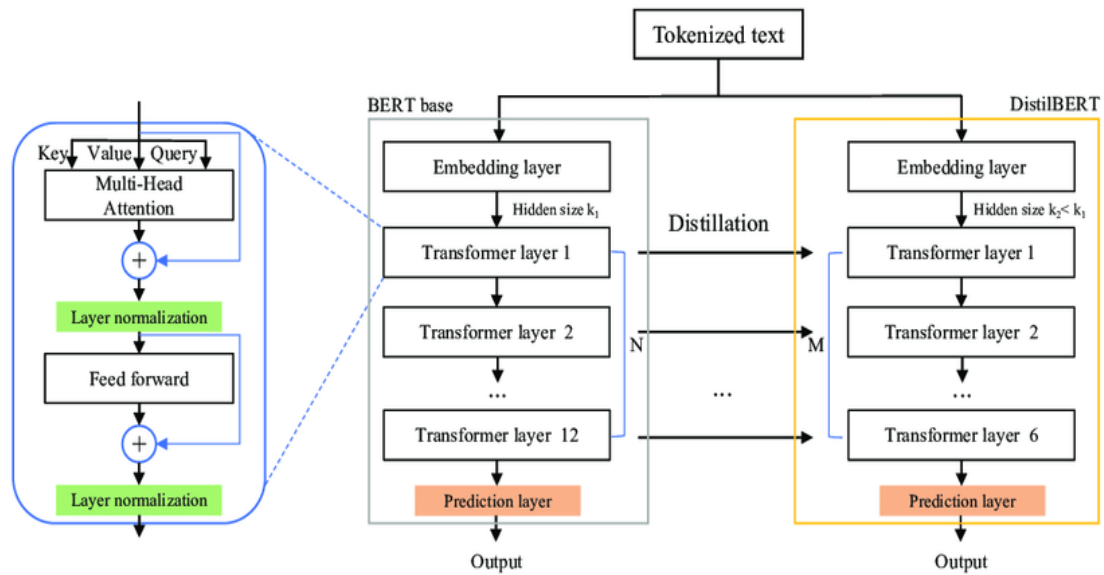


Figure 5.2: Working of DistilBERT

The DistilBERT pre-trained model is used by the suggested system to train the data. DistilBERT is a Transformer model that is compact, lightweight, quick, inexpensive, and trained by distilling BERT basis. Compared to google-BERT/BERT-base-uncased, it uses 40% less arguments and executes 60% quicker, all while maintaining over 95% of BERT's performance as determined by the GLUE language comprehension benchmark. With comparable performance, DistilBERT seeks to offer a version of BERT that is more computationally efficient. This is accomplished by using a method known as "knowledge distillation," in which a smaller model is trained to emulate the actions of a bigger model. DistilBERT, on the other hand, has a much lower parameter set and a quicker inference

time while learning to replicate the representations produced by the original BERT model.

5.2.2 Implementation of Grammatical Error Detector

The grammatical error detection module makes use of the pre-trained DistilBERT model to detect grammatical errors. The model is fine-tuned using a custom dataset of 45,798 sentences in present and future tense with labels 0 and 1 indicating correct and incorrect sentences. The grammatical error detection module takes a sentence as input and gives a result indicating whether the subject-verb agreement criterion is met or not.

5.3 GRAMMATICAL ERROR CORRECTION USING RULE-BASED APPROACH

In grammatical error correction based on a rule-based approach for Kannada, particular emphasis is placed on rectifying subject-verb agreement errors. Subject-verb agreement is a fundamental aspect of grammar that ensures that the subject and verb in a sentence agree in number and person. This agreement is crucial for the clarity and coherence of the sentence.

This rule-based approach utilizes linguistic rules and patterns specific to Kannada to identify and correct subject-verb agreement errors. It involves identifying the subject and verb and applying appropriate rules to ensure agreement between them.

For example, in Kannada, when the subject is singular, the verb must also be singular, and when the subject is plural, the verb should be plural as well.

An example illustrating subject-verb agreement in Kannada:

- Singular subject: "ಅವನು ಹೋಗುತ್ತಾನೆ" (avanu hōguttāne) - "He goes."

- Plural subject: "ಅವರು ಹೋಗುತ್ತಾರೆ" (avaru hōguttāre) - "They go."

In the above examples, the verb "ಹೋಗುತ್ತಾನೆ" (hōguttāne) agrees with the singular subject "ಅವನು" (avanu) and the verb "ಹೋಗುತ್ತಾರೆ" (hōguttāre) agrees with the plural subject "ಅವರು" (avaru).

5.3.2 Implementation of Grammatical Error Corrector

The grammatical error correction module is designed using a rule-based approach. It focuses on present and future tenses to provide suggestions. It identifies the subject and verb by stemming each word and comparing it against a list of frequently used common nouns and verbs of Kannada. It then suggests verb corrections appropriate to the count, gender, and person of the subject.

5.4 USER INTERFACE

The user interface was designed to accept user input, pass it on to the model, and display output. The front-end was developed using html, CSS and Bootstrap. The backend was developed using Flask, which is a popular Python framework.

5.4.1 Handling User Data

- Web App
Obtain input text from the user from the text editor or as an input file. The received user input is further processed.
- API Endpoint Creation
Setting up a backend server API endpoint to receive text from the web application. The text processing and transfer will be managed by this endpoint.
- Text Transfer to Backend
On the backend, the API request is to be received. The text is to be extracted from the request payload. The necessary text processing is to be applied in the backend server using NLP techniques.
- Return Corrected Text to the User
Send the corrected text as response from the backend API.

Chapter 6

RESULTS AND DISCUSSION

6.1 SPELL-CHECKER USING HUNSPELL

The Kannada spell-checker underwent testing with a sample of 20 randomly selected words containing errors. Out of these, expected corrections were provided for 16 words, resulting in an accuracy rate of 80%. This indicates that the spell-checker module has successfully identified and rectified errors in most of the tested kannada words.

6.2 GRAMMATICAL ERROR DETECTION

The table below is a classification report, which is a way to evaluate the performance of an algorithm that classifies data points into different categories. In grammatical error detection, the two categories are sentences with grammatical errors and sentences without grammatical errors.

The Kannada grammatical error detection model was trained using DistilBERT. The table shows the performance of the model that was trained to detect grammatical errors in Kannada text. It was able to achieve an accuracy of 92% on the test set containing 5088 sentences.

	precision	recall	f1-score	support
0	0.80	0.44	0.56	539
1	0.93	0.99	0.96	4041
accuracy			0.92	4580
macro avg	0.87	0.71	0.76	4580
weighted avg	0.91	0.92	0.91	4580

Figure 6.1: Grammatical error detector classification report

Here is a breakdown of the table:

Precision: is the fraction of sentences that the model classified as having grammatical errors that actually did have grammatical errors.

- Class 0 (Correctly Classified Sentence): Precision for class 0 is 0.80. This means that out of 539 sentences the model classified as grammatically correct, 80% were grammatically correct according to the test set.
- Class 1 (Incorrectly Classified Sentence): Precision for class 1 is 0.93. This means that out of 4041 sentences the model classified as having grammatical errors, 93% had errors according to the test set. In other words, the model was good at identifying sentences with errors from the ones it classified as having errors.

Recall: Recall is the fraction of sentences with grammatical errors that the model correctly classified as having grammatical errors.

- Class 0 (Correctly Classified Sentence): Recall for class 0 is 0.44. This means that out of all the grammatically correct sentences in the test set, the model was only able to correctly identify 44% of them.
- Class 1 (Incorrectly Classified Sentence): Recall for class 1 is 0.99. This means that out of all the sentences with grammatical errors in the test set, the model was able to correctly identify 99% of them. The model did a very good job of finding actual grammatical errors.

F1-Score: F1-Score is a harmonic mean of precision and recall. It is a way to balance the model's precision and recall.

- Class 0 (Correctly Classified Sentence): F1-score for class 0 is 0.56.
- Class 1 (Incorrectly Classified Sentence): F1-score for class 1 is 0.96.

Support: Support is the number of sentences in each class. In this case, there are 539 sentences classified as grammatically correct (class 0) and 4041 sentences classified as having errors (class 1) in the test set.

Accuracy: Accuracy is the total number of correctly classified sentences divided by the total number of sentences. Overall accuracy is 0.92, which means the model classified 92% of the sentences correctly.

Macro Avg: Macro Avg is the unweighted average of the precision, recall, and F1-score across the two classes.

- Macro average precision is 0.87, macro average recall is 0.71, and macro average F1-score is 0.76.

Weighted Avg: Weighted Avg is the weighted mean of the metrics across the two classes, where the weight for each class is the number of sentences in that class.

- Weighted average precision, recall, and F1-score are all very similar to macro average because there is a class imbalance (many more class 0 examples than class 1).

In this case, the model has a higher recall than precision. This means that the model is better at catching all the grammatical errors in the text than it is at avoiding incorrectly classifying correct sentences as having errors.

Overall, the table shows that the BERT model is good at detecting grammatical errors in Kannada text.

6.3 GRAMMATICAL ERROR CORRECTION

The Kannada grammatical error correction module was tested with a sample of 20 randomly chosen sentences in present and future tense, containing errors. Out of these, expected corrections were provided for 18 sentences, resulting in an accuracy rate of 90%.



Figure 6.2: Home page

The Figure. 6.2 shows a Kannada Spell and Grammar Checker application or tool. It has three main sections:

1. **Spell Check:** This section allows users to type text or upload a file to check the spelling of words in the Kannada language.
2. **Grammar error detection:** In this section, users can type or upload text, and the tool will detect any grammatical errors in the Kannada text.
3. **Grammar Check:** Similar to the grammar error detection section, this allows users to input text, and the tool will correct any grammatical mistakes found in the Kannada text.

The interface provides text boxes for entering or pasting content, buttons to initiate the spell check or grammar check processes, and an option to upload files.

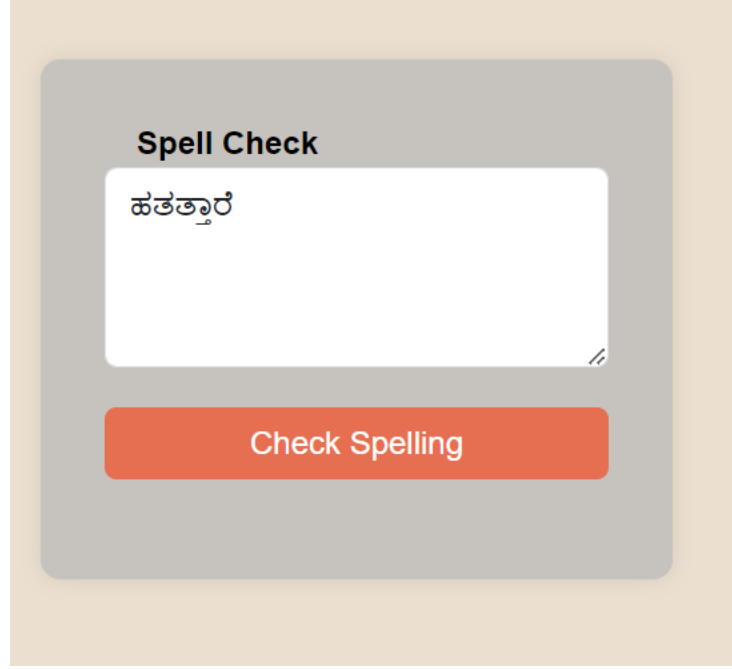


Figure 6.3 - Spell-checker input

The Figure. 6.3 shows the "Spell Check" section of the Kannada Spell and Grammar Checker application. In the text box, the word "ಹತ್ತಾರೆ" (hatattare) has been entered, which is misspelled.

This text have been entered to test the spell-checking functionality of the tool. Below the text box, there is a "Check Spelling" button which, when clicked, would analyze the entered text for any spelling mistakes in the Kannada language.

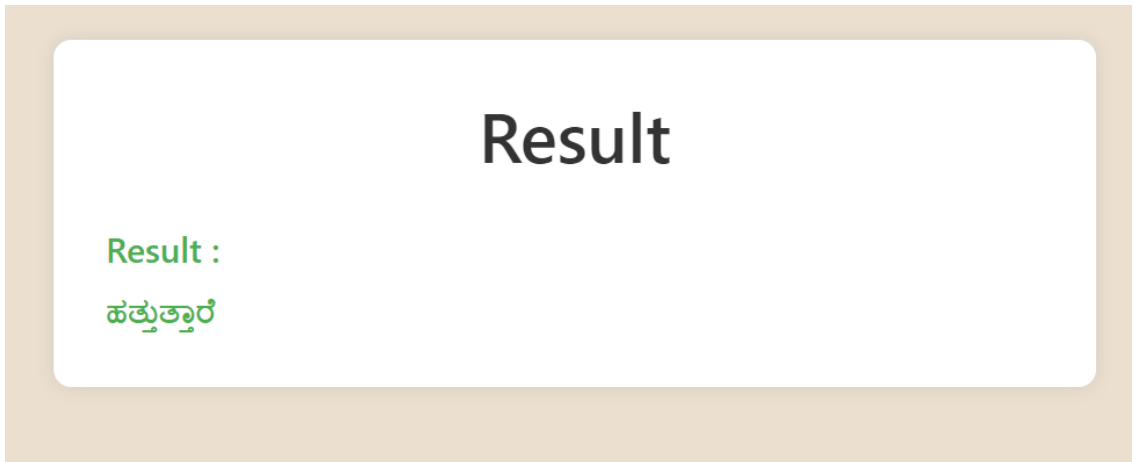


Figure 6.4 - Spell-checker output

The Figure 6.4 shows the "Result" section of what appears to be a Kannada language spell checker tool. The Result text box displays the word "ಹತ್ತತ್ತಾರೆ" (hattuttare) in the Kannada script.

Based on our previous discussion, this indicates that the spell checker has analyzed the word entered previously, which was " ಹತತ್ತಾರೆ " (hatattare), and found it to be misspelled in the Kannada language. Therefore, the result is displaying the correct form the word which is "ಹತ್ತತ್ತಾರೆ" , suggesting that spelling errors were detected.

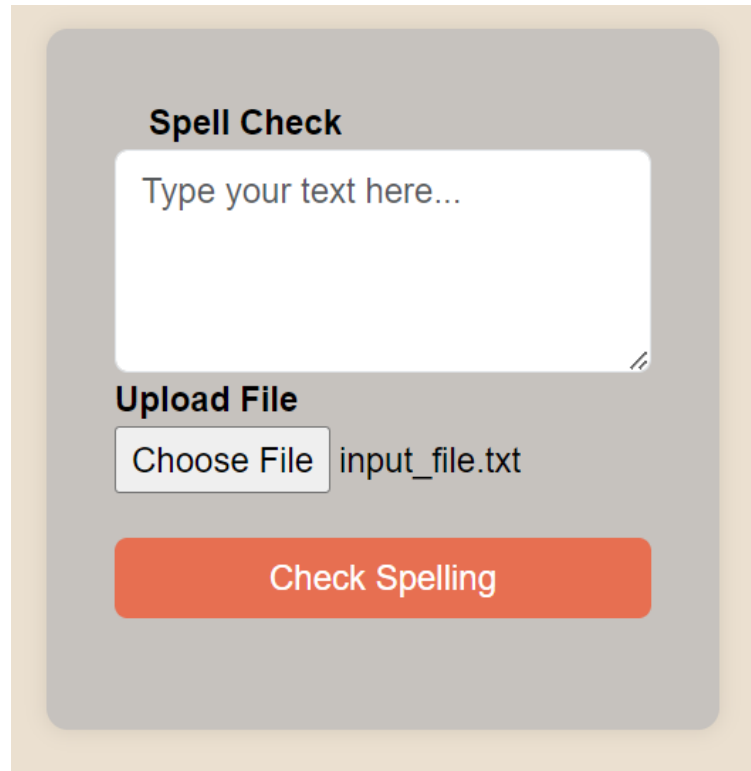


Figure 6.5 - Spell-checker file input

Figure. 6.5 shows the "Spell Check" section of which appears to be a language tool. It provides two options for checking spelling:

1. Text Input: Users can type or paste text directly into the provided text box labeled "Type your text here..."

2. File Upload: There is an "Upload File" section with a "Choose File" button. The text "input_file.txt" is shown, indicating that a file named "input_file.txt" has been selected for uploading.

Once the user has either entered text or selected a file, they can click the "Check Spelling" button to initiate the spell-checking process on the provided content.

This interface is designed to allow users to conveniently check the spelling of either typed/pasted text or content from an uploaded file, making it versatile for different use cases.

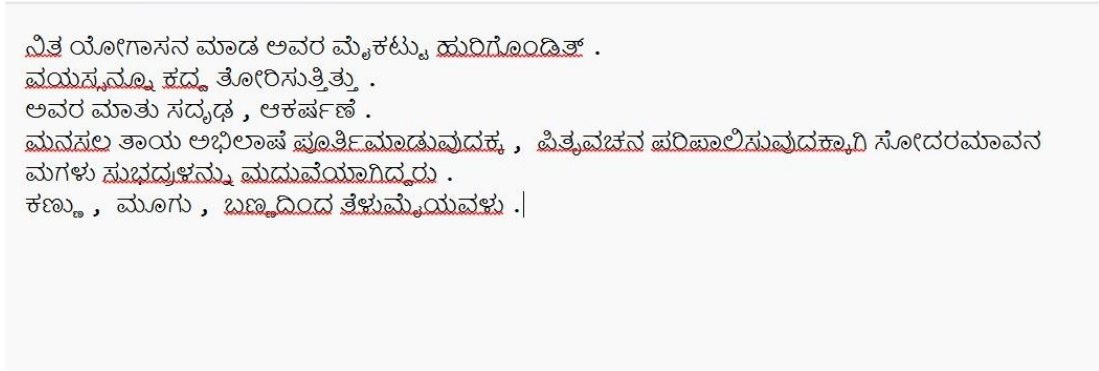


Figure 6.6 – Input file of spell-checker

Figure. 6.6 shows the content inside the "input_file.txt". The file is given to the spell checker for correction since the text file contains spelling mistake.

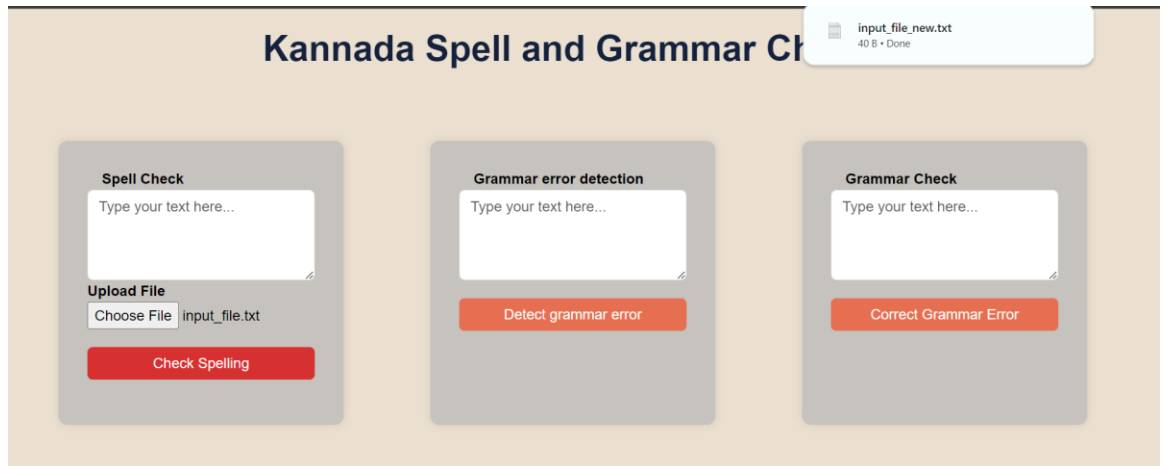


Figure 6.7 - Spell-checker file download

Figure 6.7 depicts a spell-checker interface focused on downloading files. Within the "Spell Check" section, the displayed file name, "input_file.new.txt," suggests the generation of a revised version of the original "input_file.txt" after spell-checking. The interface generates a corrected version of the input text, available for download as a new .txt file.

ನಿತ [ನಿತ್ಯ] ಯೋಗಾಸನ ಮಾಡ [ಮಾಡು] ಅವರ ಮೈಕಟ್ಟು [ಮೈ ಕಟ್ಟು] ಹುರಿಗೊಂಡಿತ್ . ವಯಸ್ಸನ್ನೂ
ಕದ್ದ [ಉದ್ದ] ತೋರಿಸುತ್ತಿತ್ತು . ಅವರ ಮಾತು ಸದೃಢ [ದೃಢ] , ಆಕರ್ಷಣೆ . ಮನಸ್ಸಲ [ಮನಸ್ಸು] ತಾಯಿ
[ತಾಯಿಯ] ಅಭಿಲಾಷೆ [ಅಭಿಮಾನಿ] ಪೂರ್ತಿಮಾಡುವುದಕ್ಕ , ಪಿತ್ತವಚನ [ಪಿತ್ತ ವಚನ]
ಪರಿಪಾಲಿಸುವುದಕ್ಕಾಗಿ ಸೋದರಮಾವನ [ಸೋಮಾರಿತನ] ಮಗಳು ಸುಭದ್ರಳನ್ನು ಮದುವೆಯಾಗಿದ್ದರು .
ಕಣ್ಣು , ಮೂಗು , ಬಣ್ಣದಿಂದ ತೆಳುಮೈಯವಳು .

Figure 6.8 – Output file of spell-checker

Figure. 6.8 which shows the output file of a Kannada language spell-checker software. The output file contains the correct word for the misspelt words the square brackets. This screenshot is being used to illustrate the software's functionality when working with Kannada text input.

Figure 6.9 - Grammatical error detector input

Figure. 6.9 showcases a user interface for a Kannada grammar error detection tool. It includes:

Title: "Grammar error detection"

Text input area with Kannada text "ಅವನು ಬರೆಯುತ್ತಾರೆ" (avanu bareyuttare), which is grammatically incorrect.

Button: "Detect grammar error"

Users can input Kannada text and click the "Detect grammar error" button to analyze it for grammatical mistakes or violations of Kannada language rules. This tool caters specifically to Kannada, with its own script and grammar rules.

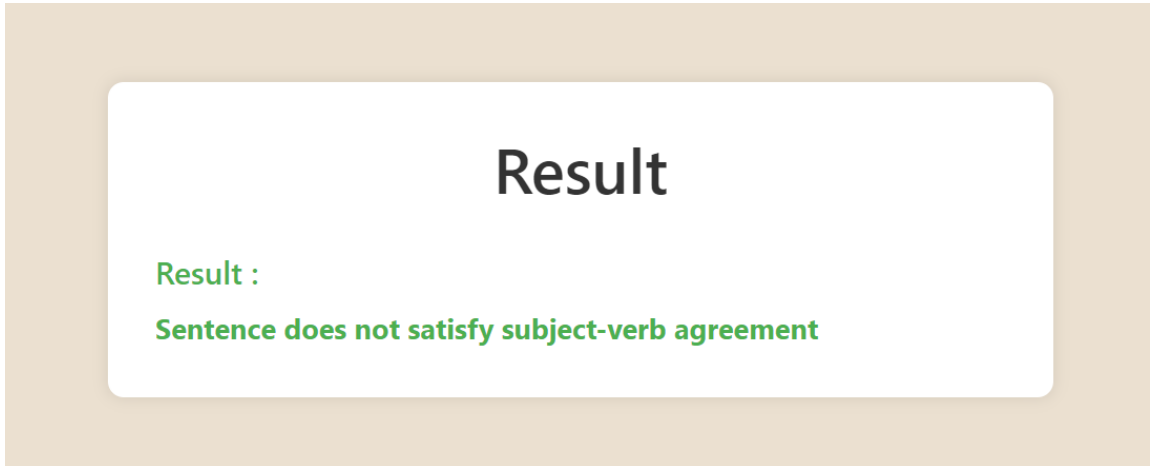


Figure 6.10 - Grammatical error detector output

Figure. 6.10 shows the output from the grammar error detection tool that was demonstrated earlier.

The output window displays the text "Result:" followed by "Sentence does not satisfy subject-verb agreement".

This result suggests that after analyzing the input Kannada sentence, the grammar checker has identified an error related to a lack of agreement between the subject and verb forms used in the sentence.

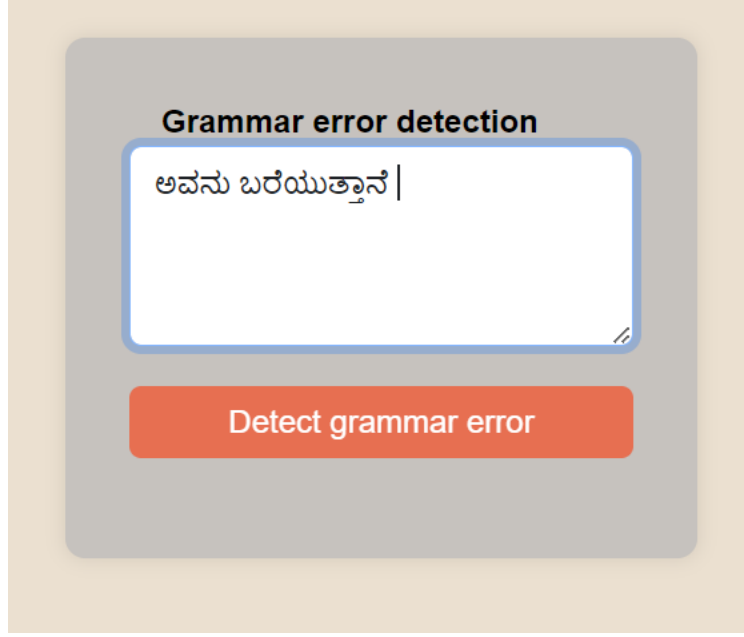


Figure 6.11 - Grammatical error detector input

Figure. 6.11 showcases a user interface for a Kannada grammar error detection tool. It includes:

Title: "Grammar error detection"

Text input area with Kannada text "ಅವನು ಬರೆಯುತ್ತಾನೆ" (avanu bareyuttane) , which is grammatically correct.

Button: "Detect grammar error"

Users can input Kannada text and click the "Detect grammar error" button to analyze it for grammatical mistakes or violations of Kannada language rules. This tool caters specifically to Kannada, with its own script and grammar rules.

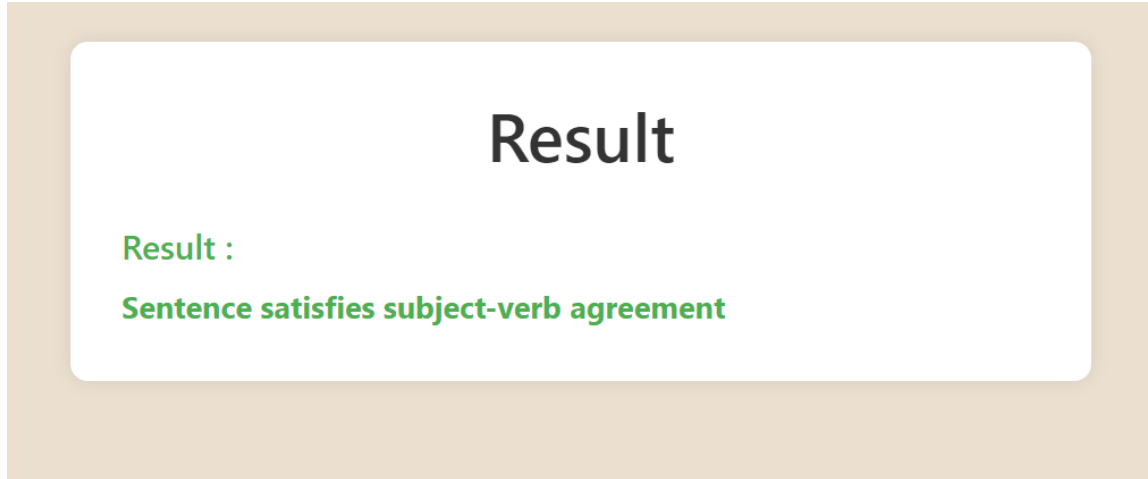


Figure 6.12 - Grammatical error detector output

Figure. 6.12 presents the output from the previously demonstrated grammar error detection tool.

The output window exhibits the text "Result:" followed by "Sentence satisfies subject-verb agreement."

This outcome indicates that upon analyzing the input Kannada sentence, the grammar checker has found no errors related to agreement between the subject and verb forms used in the sentence.

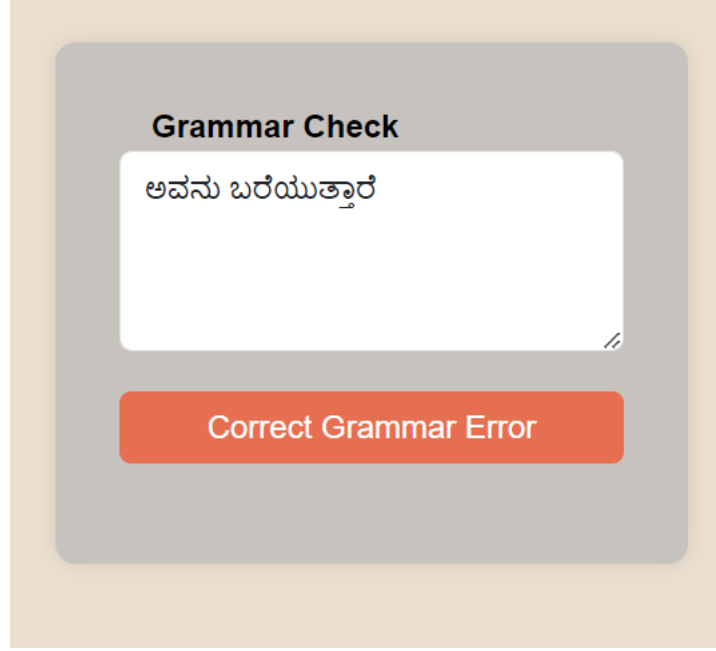


Figure 6.13 - Grammatical error corrector input

Figure 6.13 displays a user interface for a grammar checking tool specifically designed for the Kannada language. The interface has a title "Grammar Check" and contains a text input area where users can enter Kannada text to be analyzed.

In this case, the text " ಅವನು ಬರೆಯುತ್ತಾರೆ " (avanu bareyuttare) has been entered in the input area, which is grammatically incorrect.

Below the input area, there is a button labeled "Correct Grammar Error". This button likely initiates the grammar checking process, analyzing the entered Kannada text for any grammatical errors or violations of language rules.

Based on the previous examples shown, it is reasonable to expect that after clicking the "Correct Grammar Error" button, the tool will identify and provide grammatically correct form of the sentence.

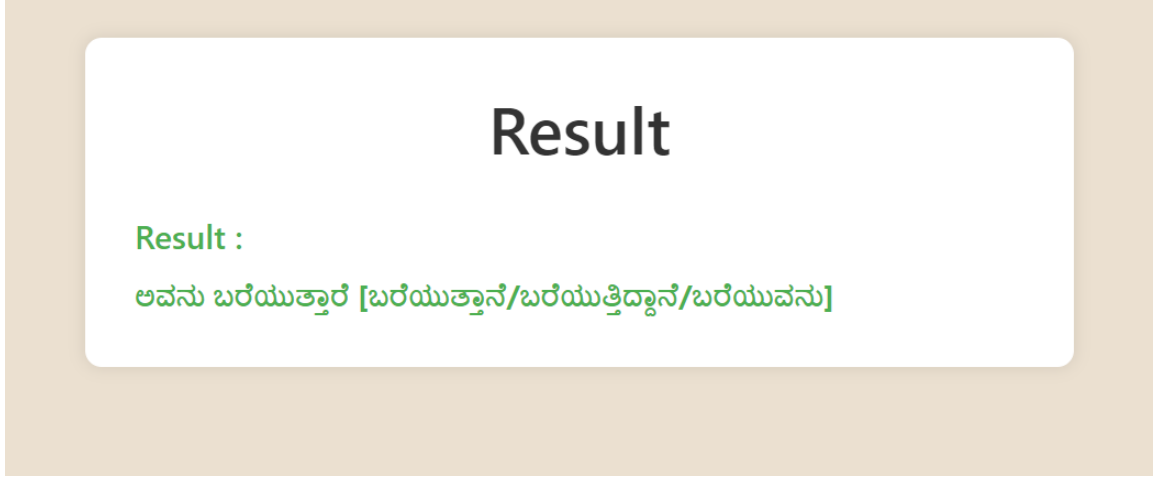


Figure 6.14- Grammatical error corrector output

Figure. 6.14 shows the result output from the Kannada grammar checking tool after analyzing the input sentence " ಅವನು ಬರೆಯುತ್ತಾರೆ " (avanu bareyuttare).

The output displays the original input sentence " ಅವನು ಬರೆಯುತ್ತಾರೆ " followed by square brackets containing three suggested corrections: "[ಬರೆಯುತ್ತಾನೆ/ಬರೆಯುತ್ತಿದ್ದಾನೆ/ಬರೆಯುವನು]".

These suggestions aim to resolve the subject-verb agreement error that was likely detected in the original sentence. The three options provided are different forms of the verb "to come" in Kannada that correctly agree with different subjects.

By providing these alternatives, the grammar tool is suggesting possible corrections to fix the subject-verb agreement issue identified in the original input sentence. The user can then choose the appropriate verb form based on the intended subject of the sentence.

Chapter 7

CONCLUSION AND FUTURE WORK

The main goal of this project is to create a Spelling and Grammar Error Correction tool for Kannada texts using the BERT model and to assess how well it works. In the past, Kannada GEC has been done using LSTM technology. In this project, we are trying a different approach using Transfer Learning. Additionally, we have added a spell checker feature to further improve the accuracy of the tool. This project aims to make the GEC process more effective and user-friendly for Kannada documents.

As we move on with our work, we will be concentrating on improving the spell and grammar-checking tool to handle irregular verb usage, varied tenses, and intricate sentence structures. In order to efficiently parse and evaluate complex sentences, our next phase of research will focus on refining algorithms and linguistic models, taking into account the dynamic nature of language and the subtleties inherent in written communication. We hope to give users more thorough feedback on sentence structure by integrating sophisticated syntactic and semantic parsing tools, guaranteeing coherence and clarity in their writing. Moreover, broadening the system's scope to include additional irregular verbs and tenses will improve the system's accuracy and applicability in many language contexts.

Through constant research, testing and iterative improvements, we are committed to making the spell and grammar-checking tool a versatile and indispensable tool for writers who want to improve their skills and communicate with precision and finesse.

REFERENCES

- [1] Saluja, R., Adiga, D., Chaudhuri, P., Ramakrishnan, G., & Carman, M. (2017, November). Error detection and corrections in Indic OCR using LSTMs. In *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)* (Vol. 1, pp. 17-22). IEEE.
- [2] Aziz, R., Anwar, M. W., Jamal, M. H., & Bajwa, U. I. (2021). A hybrid model for spelling error detection and correction for Urdu language. *Neural Computing and Applications*, 33, 14707-14721.
- [3] Gupta, P. (2020, February). A context-sensitive real-time Spell Checker with language adaptability. In *2020 IEEE 14th International Conference on Semantic Computing (ICSC)* (pp. 116-122). IEEE.
- [4] Azmi, A. M., Almutery, M. N., & Aboalsamh, H. A. (2019). Real-word errors in Arabic texts: A better algorithm for detection and correction. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 27(8), 1308-1320.
- [5] Faisal, A. M. F., Rahman, M. A., & Farah, T. (2021, July). A rule-based bengali grammar checker. In *2021 Fifth World Conference on Smart Trends in Systems Security and Sustainability (WorldS4)* (pp. 113-117). IEEE.
- [6] Gondaliya, Y., Kalariya, P., Panchal, B. Y., & Nayak, A. (2022). A Rule-based Grammar and Spell Checking.
- [7] Pabasara, H. M. U., & Jayalal, S. (2020, September). Grammatical error detection and correction model for Sinhala language sentences. In *2020 International Research Conference on Smart Computing and Systems Engineering (SCSE)* (pp. 17-24). IEEE.
- [8] Hao, S., & Hao, G. (2020, November). A research on online grammar checker system based on neural network model. In *Journal of Physics: Conference Series* (Vol. 1651, No. 1, p. 012135). IOP Publishing.
- [9] Wang, Q., & Tan, Y. (2020, July). Grammatical error detection with self attention by pairwise training. In *2020 International Joint Conference on Neural Networks (IJCNN)* (pp. 1-7). IEEE.
- [10] Murugan, S., Bakthavatchalam, T. A., & Sankarasubbu, M. (2020). SymSpell and LSTM based Spell-Checkers for Tamil.

